# DDCA PROJECT

# Design a binary to Gray code converter and explain its working and applications.

## TEAM MEMBERS:


- Anjana Varma - 2620030395
- Divija Laxmi - 2620030338
- Ganesh Goud - 2620030371
- Bindu Sri - 2620030264

# Problem Statement

- Binary code is widely used in digital systems to represent numbers and data.
- When binary numbers change from one value to the next, multiple bits may change simultaneously.
- This can cause temporary incorrect readings due to differences in signal timing.
- Gray code reduces this problem because only one bit changes between consecutive values.
- Therefore, this project aims to design a Binary to Gray Code Converter using combinational logic circuits.

# Introduction

- Binary code represents numbers using only 0 and 1.
- Gray code is a binary coding system in which consecutive numbers differ by only one bit.
- It helps reduce errors caused by multiple bits changing simultaneously.
- It is used in digital electronics and position-encoding systems.
- A Binary to Gray Code Converter converts a binary input into its equivalent Gray code output.

**Example:**

| Decimal | Binary | Gray Code |
|---|---|---|
| 0 | 000 | 000 |
| 1 | 001 | 001 |
| 2 | 010 | 011 |
| 3 | 011 | 010 |

# Background Study

**Binary Code:**

- Uses 0 and 1 to represent numbers.
- Multiple bits may change when moving between consecutive values.
- Example: 0111 → 1000 changes all four bits.

**Gray Code:**

- Only one bit changes between consecutive values.
- Reduces errors caused by simultaneous bit transitions.
- Example: 0100 → 1100 changes only one bit.

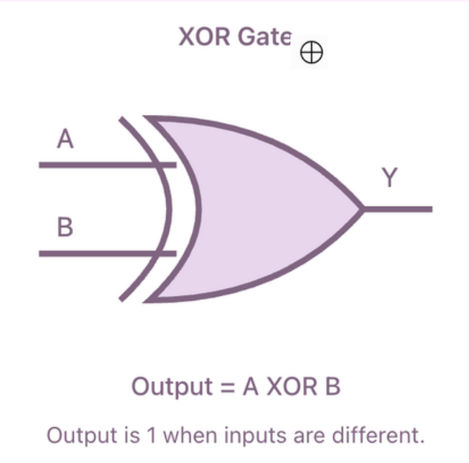
**Conversion Method:**

- Gray code is generated by keeping the MSB unchanged and applying XOR operations to adjacent binary bits.

# Design/Methodology

- For a 4-bit binary input B3,B2,B1,B0 the output equations are:

$$G3 = B3$$
$$G2 = B3 \oplus B2$$
$$G1 = B2 \oplus B1$$
$$G0 = B1 \oplus B0$$




**Explanation:**

- The most significant bit is directly connected to the output.
- The remaining outputs are generated using XOR gates.
- The circuit requires 3 XOR gates and 1 direct connection.

# Circuit Diagram

## Circuit Diagram of Binary to Gray Code Converter:

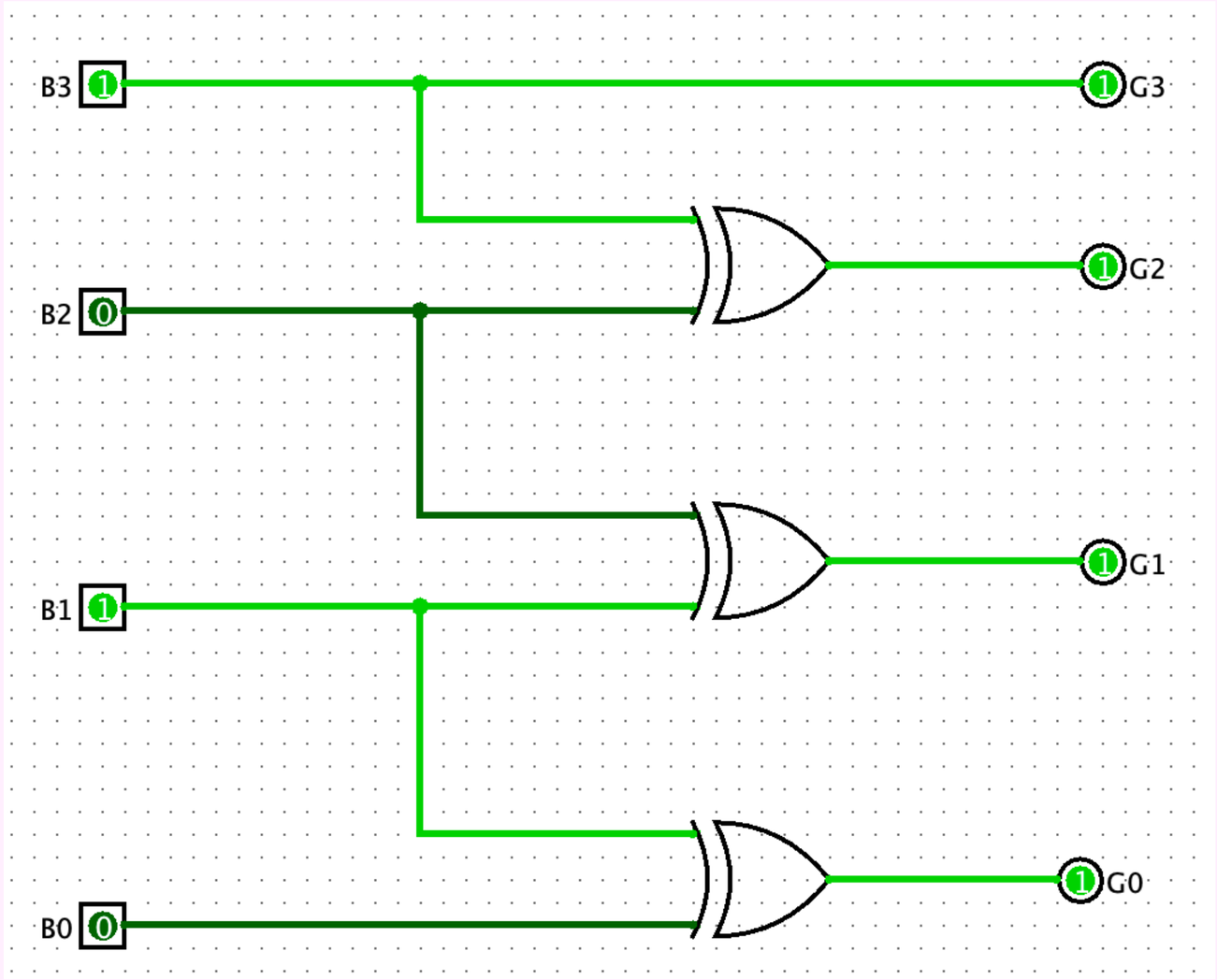
The 4-bit binary-to-Gray code converter is designed using XOR gates and a direct connection. The circuit takes four binary inputs and produces four Gray code outputs.

## Circuit Connections:

| Binary input(s) | Operation | Gray output |
|---|---|---|
| B3 | Direct connection | G3 |
| B3, B2 | XOR gate | G2 |
| B2, B1 | XOR gate | G1 |
| B1, B0 | XOR gate | G0 |

- Four input pins on the left: B3, B2, B1, B0.
- Three XOR gates in the middle.
- Four output pins on the right: G3, G2, G1, G0.
- Connect the inputs according to the table.

# Circuit Diagram – Logisim Implementation.



# Truth Table

## Truth Table and Working Principle:

The truth table shows the conversion of all 16 possible 4-bit binary inputs into their corresponding Gray code outputs. The most significant bit remains.  unchanged, and each remaining Gray bit is obtained by XOR-ing two adjacent binary bits.

**For binary input 1010:**

- **G3 = 1**
- **G2 = 1 XOR 0 = 1**
- **G1 = 0 XOR 1 = 1**
- **G0 = 1 XOR 0 = 1**

**Therefore, Binary 1010 → Gray code 1111.**

| Binary input | Gray output |
|---|---|
| 0000 | 0000 |
| 0001 | 0001 |
| 0010 | 0011 |
| 0011 | 0010 |
| 0100 | 0110 |
| 0101 | 0111 |
| 0110 | 0101 |
| 0111 | 0100 |
| 1000 | 1100 |
| 1001 | 1101 |
| 1010 | 1111 |
| 1011 | 1110 |
| 1100 | 1010 |
| 1101 | 1011 |
| 1110 | 1001 |
| 1111 | 1000 |

# Observation

- All 16 possible 4-bit binary inputs were verified.
- The outputs match the expected Gray code values.
- The most significant bit remains unchanged.
- Only one bit changes between consecutive Gray code values.

| B3 | B2 | B1 | B0 | G3 | G2 | G1 | G0 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |

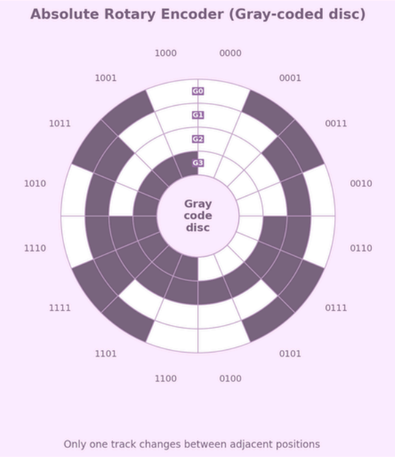
# Applications

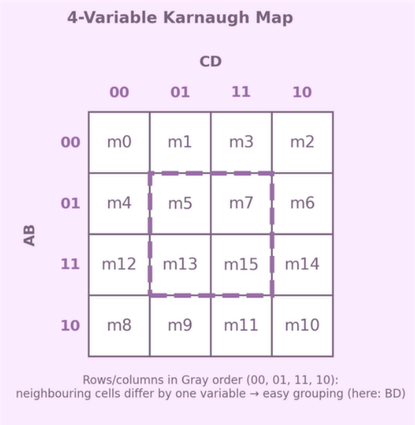
**Rotary/optical encoders:** only one bit changes per step, so a misaligned read is off by at most one position instead of giving a wildly wrong value (e.g. 0111 → 1000 could be misread as 1111).

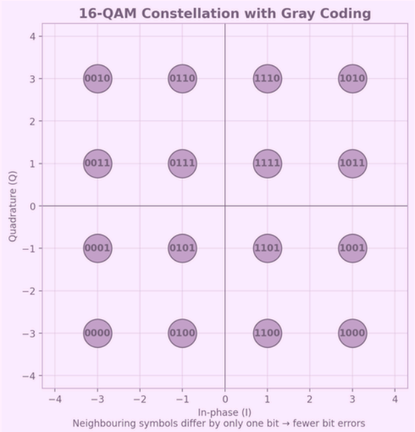
**Karnaugh maps:** rows and columns are ordered in Gray code so adjacent cells differ by one variable.

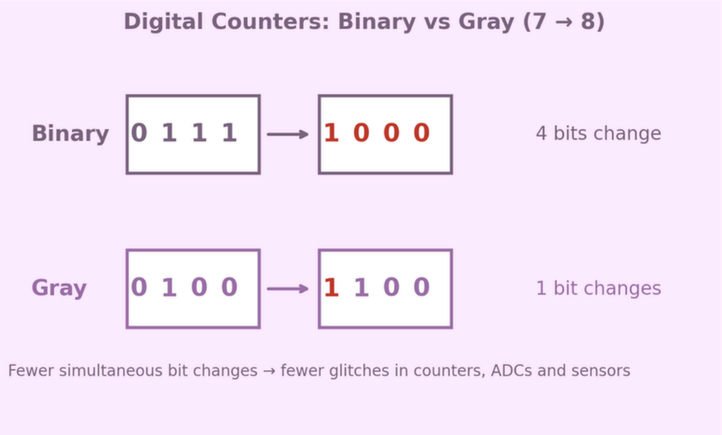
**Digital communication:** Gray-coded symbol mapping in modulation (QAM/PSK) reduces bit errors, and async FIFO pointers use Gray code for safe clock-domain crossing.

**Digital counters:** fewer simultaneous transitions mean fewer glitches.



# Conclusion

The 4-bit Binary-to-Gray Code Converter was designed using 3 XOR gates and 1 direct connection and simulated in Logisim. All 16 input combinations were verified using the truth table. The circuit produces the correct Gray code output, ensuring that consecutive Gray code values differ by only one bit. This demonstrates the practical application of Boolean logic in digital circuit design.

# GitHub Repository

**Repository name:** Binary-to-Gray-Code-Converter

**Files included:**

- **README.md** — project explanation and Boolean equations.
- **binary_to_gray.circ** — Logisim circuit file
- images/ — circuit and simulation screenshots

**GitHub Repository URL**:https://github.com/KLH-Parity-Generator/KLH-FED-2026-Session13-Batch-6

# Thank You

Any Question?